PRÁCTICA 3.1
AGRUPACIÓN DE CLIENTES

Segmentación de 524 clientes según sus hábitos de compra y gasto, con K-Means en Python y RapidMiner.

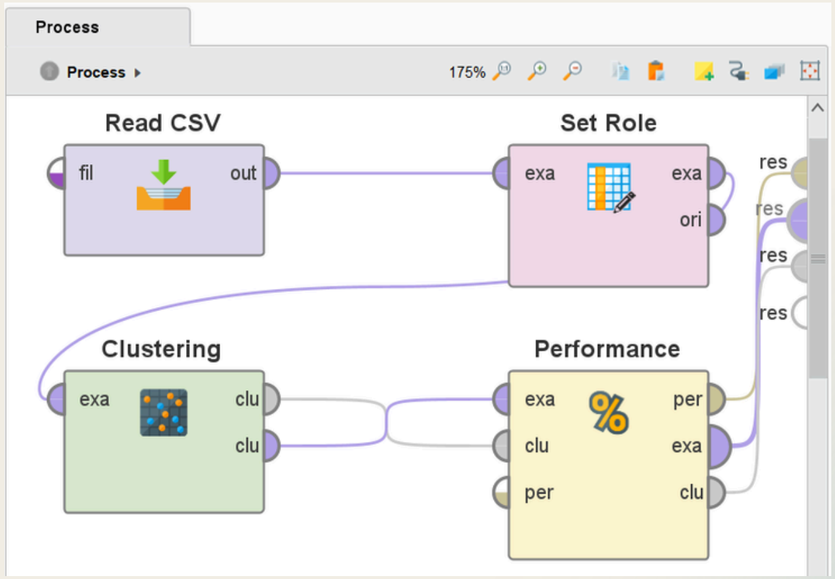
Equipo 7
· Flores García Claudio
· Juárez Valdivia Barvara Caridad
· Ortega de la Paz Rafael

PROCESOS EN RAPIDMINER

Datos limpios de Python

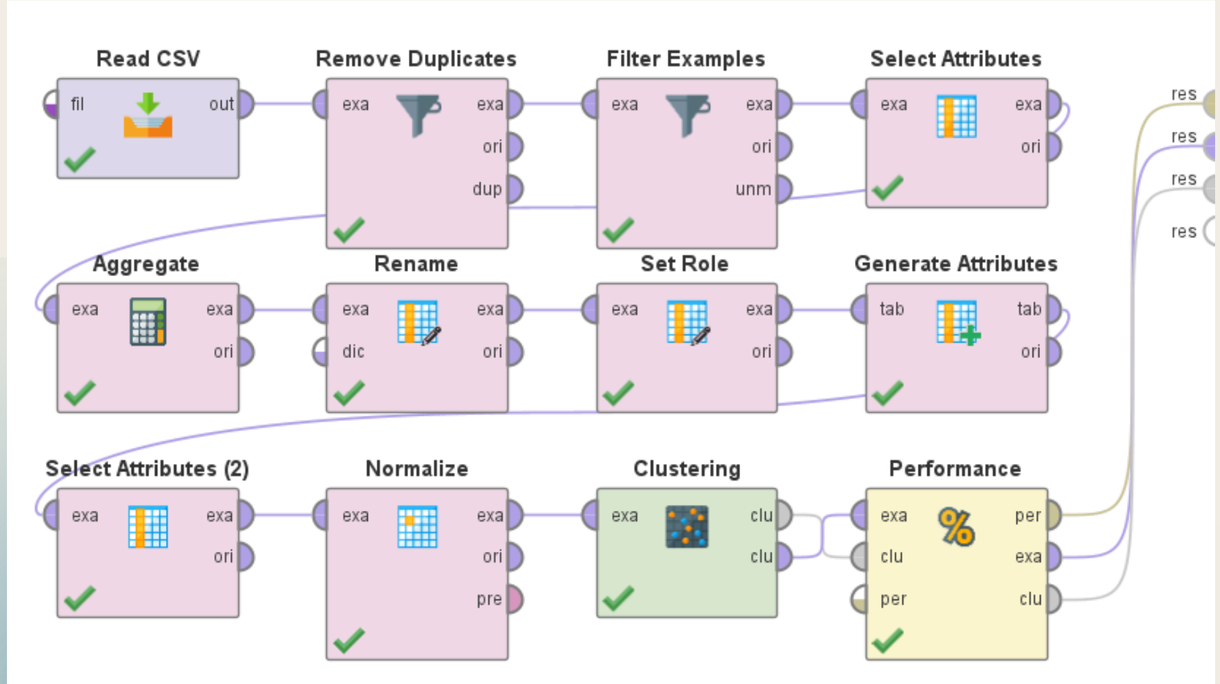
Se carga el CSV con las 8 variables ya limpias, escaladas y transformadas (líneas de venta agregadas por cliente), y se aplica K-Means.

k-Means: k=3, distancia euclidiana, 8 variables por cliente

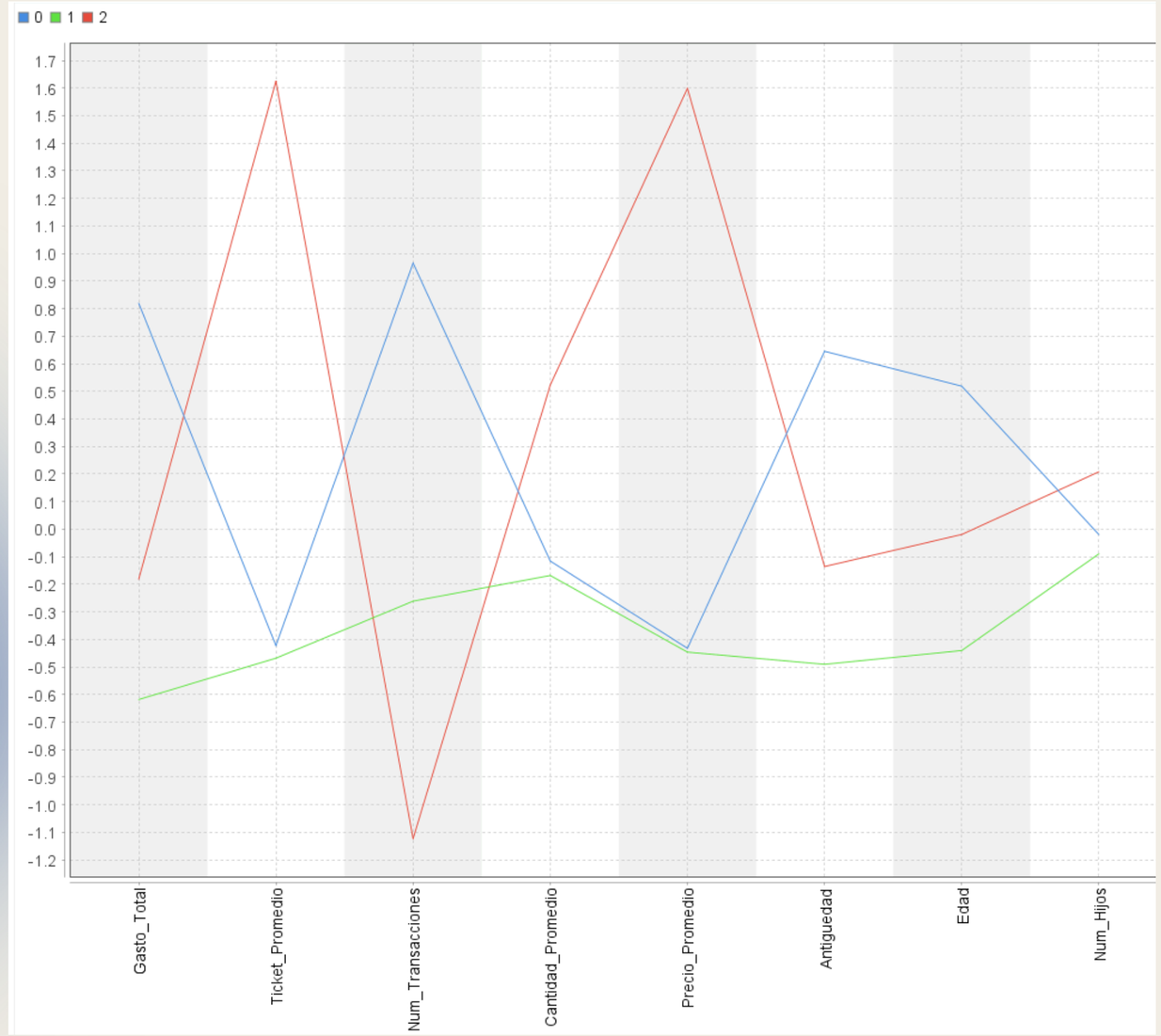


Desde el archivo original

Desde el CSV original -> (58,903 líneas de venta) hasta K-Means: limpieza, agregación por cliente (líneas → 524 clientes en 8 variables), logaritmo en variables sesgadas, estandarización y clustering, todo en RapidMiner.



SEGMENTOS DE CLIENTES



cluster_0 · Leales de alto valor – 191 Clientes
Más líneas de venta (+0.97) y gasto (+0.82), mayor antigüedad (+0.65) y edad (+0.52).

cluster_1 · Jóvenes de bajo gasto – 220 Clientes
Todo por debajo del promedio: gasto (-0.62), antigüedad (-0.49) y edad (-0.44).

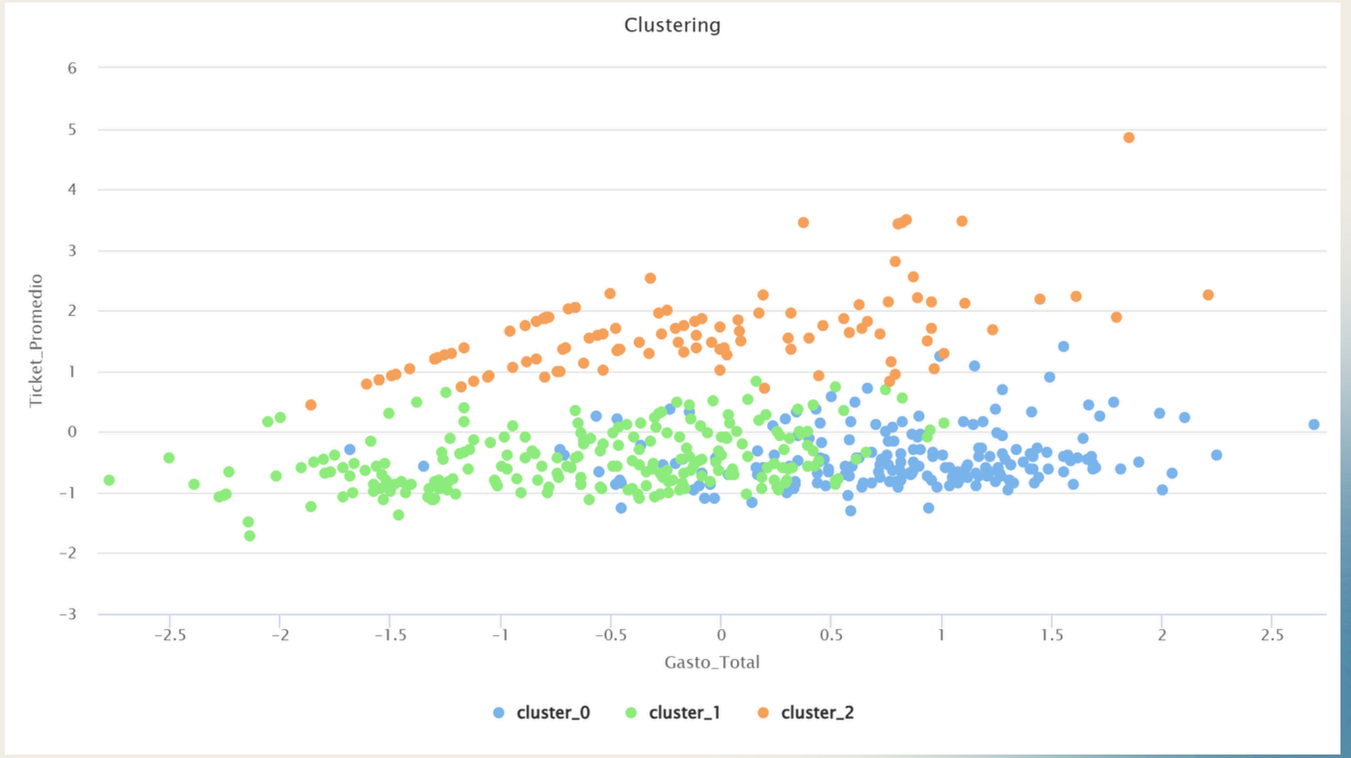
cluster_2 · Ticket alto – 113 Clientes
Ticket (+1.63) y precio promedio (+1.60) muy altos, pero pocas compras (-1.12).

Centroid Plot: el perfil de cada cluster en las 8 variables.

"Nota Se utilizaron 3 componentes principales (que explican 74.8% de la varianza total de las 8 variables). K-Means sobre PCA produce casi la misma segmentación (índice Adjusted Rand Index ≈ 0.91)."

GASTO TOTAL VS TICKET PROMEDIO

El ticket alto (naranja) se separa arriba, leales (azul) y bajo gasto (verde) se distinguen por el gasto.



MISMOS CENTROIDES POR AMBOS CAMINOS

Attribute	cluster_0	cluster_1	cluster_2
Gasto_Total	0.819	-0.618	-0.181
Ticket_Promedio	-0.422	-0.468	1.626
Num_Transacciones	0.965	-0.261	-1.123
Cantidad_Promedio	-0.116	-0.168	0.523
Precio_Promedio	-0.432	-0.446	1.599
Antiguedad	0.645	-0.491	-0.136
Edad	0.519	-0.440	-0.019
Num_Hijos	-0.020	-0.090	0.208

Proceso 1 · Datos limpios de Python

Attribute	cluster_0	cluster_1	cluster_2
Antiguedad	0.645	-0.490	-0.136
Edad	0.518	-0.440	-0.019
Num_Hijos	-0.020	-0.090	0.208
log_Gasto	0.819	-0.618	-0.181
log_Ticket	-0.422	-0.468	1.624
log_Transacciones	0.964	-0.261	-1.122
log_Cantidad	-0.116	-0.168	0.522
log_Precio	-0.432	-0.446	1.598

Proceso 2 · Desde el archivo original

VALIDACIÓN Y CONCLUSIONES

PerformanceVector

```
PerformanceVector:
Avg. within centroid distance: -4.993
Avg. within centroid distance_cluster_0: -4.355
Avg. within centroid distance_cluster_1: -3.869
Avg. within centroid distance_cluster_2: -8.258
Davies Bouldin: -1.473
```

Proceso 1 · Datos limpios de Python

PerformanceVector

```
PerformanceVector:
Avg. within centroid distance: -4.983
Avg. within centroid distance_cluster_0: -4.346
Avg. within centroid distance_cluster_1: -3.862
Avg. within centroid distance_cluster_2: -8.242
Davies Bouldin: -1.473
```

Proceso 2 · Desde el archivo original

· RapidMiner hace el proceso completo
Desde el archivo original hasta la validación, sin Python, y con el mismo resultado: mismos grupos y Davies-Bouldin de ***1.473***

· PCA para interpretación.
Las 3 primeras componentes principales explican 74.8% de la varianza. PC1 (32.4%) resume consumo/frecuencia; PC2 (24.7%), fidelidad/madurez; PC3 (17.7%), perfil familiar

· K-Means fue el mejor modelo
En Python ganó en Silhouette (0.224), Davies-Bouldin (1.473, menor es mejor) y Calinski-Harabasz (157, mayor es mejor) frente a DBSCAN (0.011, 29% ruido) y jerárquico (0.177). RapidMiner replica estos números exactos.

· Una estrategia por segmento
Lealtad para los de alto valor, activación para los jóvenes de bajo gasto y promociones premium para los de ticket alto.

¡GRACIAS POR SU ATENCIÓN!